

Emerging Threats PRO/OPEN Ruleset for Suricata 7.0.3 Now Available

Rich argonzo

Jun 2024

Greetings! The ET team is pleased to announce formal support for Suricata 7.0.3! Starting with last night's rule push we've forked our existing ruleset and are offering those rules for download to our customers (ETPRO) and the Community (ET Open).

How to Download the new Ruleset

The instructions for acquiring this new ruleset are nearly identical to previous versions of the Suricata ruleset. For those who manually download the ETOPEN ruleset from rules.emergingthreats.net (via `curl`, `wget`, or their web browser of choice), the only thing that is changing is the version string for the version of Suricata rules you want to download. For example, using `wget` to download the Suricata 7.0.3 `emerging-all.rules.tar.gz` file would look like:

```
wget http://rules.emergingthreats.net/open/suricata-7.0.3/emerging-all.rules.tar.gz
```

For ETPRO users, your `wget` command will also change slightly:

```
wget https://rules.emergingthreatspro.com/[oinkcode here]/suricata-7.0.3/etpro-all.rules.tar.gz
```

meanwhile, users of `suricata-update` running suricata 7.0.3 or later should get the new ruleset automatically the next time `suricata-update` is run. If you have a reason to avoid this and wish to continue using the Suricata 5 ruleset, I would recommend utilizing the flag `--suricata-version 5.0.0` as a part of your `suricata-update` command.

Changes in the 7.0.3 ruleset

In practice, this effort has entailed a number of changes to our production 5/6 rulesets. Thanks to critical features added in Suricata 7.0.3 we made the decision to massively uplift those existing rules into this new offering. Let's talk about why we made that call.

For Suricata 7.0.3 OISF made feature additions that required us to modify existing rule syntax in order for those affected rules to work with both HTTP/1 and HTTP/2.

HTTP2

In our testing and analysis, there were many reports of False Positives and False Negatives due to the character case differences of HTTP header names between HTTP

versions. As a result of this behavior, better support for HTTP/2 was the highest priority effort in this rule fork, and the single largest change to the body of our rules.

The transformations and modification of rule contents to better support HTTP/2 traffic impacted over 9,500 rules. Below is a summary of the many considerations taken to determine which rules would be modified to support HTTP/2 traffic and which would be considered an “http1 only” rule.

Approximately 2800 rules were considered an “HTTP1 only” rule when any of the following conditions were met:

- Detection logic which used the HTTP Header Name `Host` which is largely supplanted by the `authority` pseudo-header (see [Addressing HTTP/2 in Suri7](#) for more details)
- Detection logic which included HTTP/1.x protocol-specific sticky buffers (e.g., `http.request_line`, `http.response_line`, `http.start`, `http.protocol`)
- Rules written prior to 2015 which required modifications due to detection logic which includes HTTP header names that would have required being converted to lowercase within `http.header` or `http.start`
- Use of the `http.start` sticky buffer, which is not populated for HTTP/2 traffic by Suricata
- Unbuffered content matches that were too ambiguous to be moved to a more specific buffer (given that headers and other parts of the request line (including the URI) are compressed within HTTP/2, unbuffered contents are likely to cause FNs within HTTP/2 traffic.)

For the remaining HTTP rules, we made the following choices when modifying the rules for usage with HTTP/2:

- Rules that specified content matches in the `http.header` or `http.header_names` sticky buffers now utilize the `header_lowercase` transformation or the `to_lowercase` transformation for the rule to trigger correctly on both HTTP/1 and HTTP/2 network traffic.
- Rules that utilize the `bsize` or `startswith` keywords within the `http.header`, `http.header.raw` and/or `http.header_names` now has the `strip_pseudoheaders` transformation applied to ensure the ability to consistently alert on HTTP/1 or HTTP/2 traffic.

Rule Category Changes

In order to best serve the analysts that catch the alerts we generate we try our best to parse-out our rules into workable (and filterable!) categories. Massive rule forks like this present an opportunity for us reorganize rules, and their categories. With this latest ruleset, we have introduced some new categories, and removed some older categories, merging their rules into either a new or existing category.

New categories:

FILE_SHARING: The FILE_SHARING category contains rules that offer detection against file sharing protocols and/or services. Think of these rules as informational rules that inform analysts about file sharing related services observed in network traffic. For example, traffic related to Dropbox services, or perhaps DNS/TLS SNI rules for visits to websites associated with the bit-torrent protocol.

While these services may be considered legitimate (Depending on the network, and network policies), these rules can be used to detect anomalies or as the basis of an investigation if certain rules in the category alerts began triggering recently, with no prior history. New IDS alerts indicate a change in activity patterns, and may be a sign of compromise, or otherwise anomalous activity that should be investigated further.

REMOTE_ACCESS: The REMOTE_ACCESS category offers detection against a variety of remote access protocols and/or visits to websites offering remote access services.

Much like the FILE_SHARING category, these services are typically considered legitimate. However, sudden IDS alerts triggering from this category that have never (or very rarely) triggered before indicate a change in activity patterns, and may warrant further investigation into the root cause.

DYN_DNS: In a nutshell, Dynamic DNS is a type of service that allows users to register a sub-domain to a leased public IP address that may change frequently. In most cases, the uses of Dynamic DNS services in an enterprise environment are very limited, while many threat actors like to utilize Dynamic DNS for various purposes – hosting phishing sites, malicious payloads, malware command and control, among other things. Again, these rules are mainly informational rules that can be used to detect a change in behavior patterns, and may require further investigation as to why that pattern changed.

TA_ABUSED_SERVICES: This new category represents rules for services that have a tendency to be abused by threat actors. Examples may include URL shorteners abused for threat actor operations, certain file sharing services, Pastebin-like services, etc. Alerts triggering from this category are NOT guaranteed to be threat actor activity, but should be investigated further, especially if it represents a change in behavior patterns or typical activity observed.

RETIRED: Decisions to “Delete” rules (that is, moved into the “DELETED” category) do not come easy. What may be considered an old and outdated threat to some may be another organization’s worst nightmare they have to keep track of. Additionally, many penetration testing firms offer “adversary emulation” services, that emulate indicators, tools, techniques, and tradecraft of certain threat actors. On top of that, many sandboxing services utilize the Emerging Threats ruleset for analyzing the network traffic of samples. Sometimes, these IDS alerts are the only way of positively ID’ing what malware family a sample belongs to. If these rules are moved to the DELETED

category, they are also disabled by default. On top of all that, once a rule is placed into the DELETED category, it is the official policy of the Emerging Threats team that the rule is no longer supported. Not every organization is going to delve through the list of rules in the DELETED category to determine which rules are still relevant for them, nor would they want to be responsible for maintaining or modify the rules if there are problems with false positives or performance.

We determined that a solution was needed, and came up with the “RETIRED” category. Rules placed into this category are a step between the “Actively Supported” of our primary rule categories, and the “No longer supported at all” of the DELETED category. Rules placed in “Retired” category are still enabled, and will still actively trigger alerts, but are supported on a “Best Effort” basis. That is, the Emerging Threats team will make reasonable efforts to fix possible detection issues – False positives, False negatives, etc. But will not receive updates to rule syntax for future rule forks. This in turn may impact the performance and/or compatibility of these rules with future versions of Suricata.

In this way, we can support our users that need rules to detect older threats enabled, but for users who are not interested in these rules, they can selectively disable rules in the retired category, or disable the category entirely.

Deprecated Rule Categories

The following rule categories have been removed, and merged into other existing rule categories:

- POLICY has been merged with INFO
- SCADA_SPECIAL has been merged with SCADA
- ICMP_INFO has been merged with ICMP

Suricata 4 Style keywords and Style Guide Changes

The ETPRO and ET Open rules represent over a decade of research, testing, authoring, and support resulting in over 100K rules. Over the years, Suricata has evolved with more features, more keywords, and new syntax require to utilize those new features. So, before the work to support the new features of Suricata 7.0.3 could be realized, we first had to modernize and normalize the ruleset making a number of usability & ‘style’ changes and choices. In this instance, we noted that there were a number of rules in the Suricata 5.0.0 ruleset that were still utilizing Suricata 4.0.0 syntax, and have made great strides towards modernizing the ruleset, and ensuring consistency between new additions to the ruleset, and our legacy rules.

With that “uplift” of the ruleset complete, focus was given to using more specific keywords when possible. For example, the following syntax:

```
http.header; content:"User-Agent|3a 20|Hello Word|0d 0a|";
```

This rule syntax is how the Emerging Threats team use to perform content matches against the http User-Agent field in Suricata 4. Compare that to the following Suricata 5 rule syntax:

```
http.user_agent; bsize:10; content:"Hello Word";
```

The use of the `http.user_agent` sticky buffer allows for a case insensitive matching of the User-Agent header name, thus better supporting both HTTP/1 and HTTP/2 traffic. In addition to `http.user_agent`, here are some other keywords we have utilized to replace Suricata 4 Keywords, and better optimize our ruleset:

- `http.host`
- `http.content_type`
- `http.referer`
- `http.content_len`
- `http.server`
- `http.connection`
- `http.location`
- `http.accept`

Enforcement of the Suricata Style Guide Standards

Several elements of the recently published [Suricata Community Style Guide](#) were also addressed. While not impacting the performance of the ruleset, the rule fork was an opportunity to make edits impacting the majority of the ruleset, without creating several thousand modifications of an actively used ruleset. As such, the following changes were applied to all non-deleted rules:

- For all TCP and TCP application protocol rules, the `flow` keyword has been normalized to:
 - `flow:established,[direction];`
 - `direction` is defined as either `to_client` or `to_server`
 - e.g., `flow:established,to_server;` or `flow:established,to_client;`
 - Where applicable, the `flow` keyword is always the first detection logic keyword (usually after the `msg` keyword)
- For rules utilizing the `flowbits` keyword, the keyword is defined immediately after the `flow` keyword
- The `threshold` keyword has been moved to the end of the detection logic (e.g., the last keyword prior to the `reference` (if present), `classtype`, `sid`, `rev`, and `metadata` keywords)
- Wherever possible, the `depth` keyword has been replaced with `startswith`
- The `bsize` keyword has been moved after sticky buffer keyword declaration
 - for example: `http.header_names; bsize:14; to_lowercase; content:"|0d 0a|hostname|0d 0a 0d 0a|";`

New Features in Suricata 7.0.3

While you could compare and contrast features from the [Suricata 5 Read the Docs](#) page and the [Suricata 7.0.3 read the docs](#) page, it doesn't answer the big question: What are some big changes that may be relevant to the future of the Emerging Threats ruleset?

Additional Transformations

While we've covered the `strip_pseudo_headers`, `to_lowercase`, and `header_lowercase` keywords as our primary method of enabling detection against HTTP1 or HTTP2 traffic interchangeably, there are a few new transformation options that have the potential to be game-changers in the future.

Transformation keywords can be used to modify payload data in a sticky buffer prior to being evaluated by Suricata. They can be combined, and are evaluated in the order in which they appear in a rule. In addition to the transformations already mentioned, Suricata 7.0.3 introduces `url_decode` and `xor`.

xor

XOR operations are frequently used by adversaries to obfuscate network traffic. If the XOR key being utilized is static, the `xor` transformation can be provided a key that can transform the sticky buffer's content and reveal the plaintext data for matching against other Suricata keywords. The syntax looks something like this:

```
[sticky buffer]; xor:"[xorkey]"; content:"ayy lmao";
```

Replace the `[sticky buffer]` placeholder with a sticky buffer the xor transform should be applied to. The `[xorkey]` value must be hexadecimal encoded. Here is an example rule, using the `http.request_body` sticky buffer, and the three-byte xor key "Ayy":

```
alert http any any -> any any (msg:"ayy lmao in request body";  
flow:established,to_server; http.method; content:"POST";  
http.request_body; xor:"415959"; content:"ayy lmao"; sid:1; rev:1;)
```

That rule, will match against the network traffic represented in the http request body in this pcap:

url_decode

By default, Suricata will automatically decode instances of URL encoding in the `http.uri` sticky buffer. This is not the default behavior for other sticky buffers. With the addition of the `url_decode` transformation, data in the specified sticky buffer will be

decoded. For example the “+” character will become a space and hex encoded values with a leading “%” symbol will be decoded to their ASCII equivalent. for example, let’s take the following value:

```
Cookie: %41%59%59
```

and apply the `url_decode` transformation to it:

```
alert http any any -> any any (msg:"ayy in cookie";  
flow:established,to_server; http.method; content:"POST"; http.cookie;  
url_decode; content:"AYY"; sid:1; rev:1;)
```

This rule will trigger on the network traffic in the http cookie header, represented in this pcap:

New Rule Header Application Protocol Support

Suricata rules allow the use of several different network application protocols in the rule header in place of the traditional `tcp`, `udp`, and `icmp` options of the past. Here are some new application layer protocols that Suricata 7.0.3 supports:

- `rfb`
- `rdp`
- `snmp`
- `bittorrent-dht`
- `tftp`

New Protocol-Specific Keywords

In addition to the new protocol options available in the Suricata rule header, there are several new application layer protocol specific keywords. Here are a few examples:

- **RFB (Remote Frame Buffer – VNC) Keywords**
- **SIP Keywords**
- **SNMP Keywords**
- **HTTP2 Keywords**

Feedback & Support

As always, thank you for your continued patronage and support. It is through our collaboration with, and vigilance of our user community that the Emerging Threats ruleset continues to succeed today. We rely heavily on our user community to be our eyes and ears, in terms of reporting new threats, as well as false positives or false negatives with our existing rules.

Please do not hesitate to contact us with your questions, concerns, or observations. We can be reached at:

<https://community.emergingthreats.net>
<https://feedback.emergingthreats.net>
email: support@emergingthreats.net
twitter: [@et_labs](#)
mastodon: [@EmergingThreats@infosec.exchange](#)

New & Unread Topics

Topic	Replies	Activity
[Suricata Webinar] See Clear NDR Community 1.0 in Action suricata , community	1	Aug 2025
Suri Oculus — Suricata Event Analysis System (Redis Pipeline + AI) and HBF Roadmap	0	Mar 26
Suricata Language Server 1.3.0 is available	0	Dec 2025
IDSTower v3.0.0 Released - Enhanced Rules Editor with Syntax Highlighting, Multi-Instance Architecture & Suricata 8 Support suricata	0	Feb 4
CGTI Lite 646 Suricata rules + cross-platform management tool for OpenClaw/AI agent threat detection (open source) rules , ips , community	0	Apr 1

Want to read more? Browse other topics in or [view latest topics](#).